

PowerShift: Leveraging Power-Aware Weight Approximations for Neural Network Acceleration

Vishesh Mishra*, Sparsh Mittal[†], Urbi Chatterjee*

*CSE Department, IIT Kanpur, India, [†]ECE Department, IIT Roorkee, India

*{vishesh, urbi}@cse.iitk.ac.in, [†] sparsh.mittal@ece.iitr.ac.in

Abstract—Power efficiency is a key challenge in deploying deep neural networks (DNNs) on resource-constrained edge devices. Techniques such as pruning and quantization often rely on power-intensive multiplications and are tightly coupled with specific datasets and architectures. We introduce *PowerShift*, a novel method that approximates INT8 quantized weights as power(s)-of-two to enable efficient bit-shifting in place of traditional multiplications. Experimental results on four neural networks demonstrate that *PowerShift* achieves higher power savings and lower degradation in inference accuracy than a recently proposed power-aware pruning technique (DAC’23). Furthermore, when compared with state-of-the-art power-of-two quantization method (TinyML’22), *PowerShift* provides upto 15% higher power savings while maintaining comparable accuracy.

Index Terms—DNN, Quantization, Energy-efficiency

I. INTRODUCTION

Deep neural networks (DNNs) have been widely adopted in various domains, including image recognition, speech recognition and natural language processing [1]. However, deploying DNNs on resource-constrained edge devices poses a significant challenge due to the high power consumption associated with multiply-and-accumulate (MAC) operations [2], [3]. The methods to mitigate the power consumption of DNNs can be broadly categorized as software-level techniques and hardware-level optimizations. At the software level, *pruning* and *quantization* are commonly employed [4]. *Pruning* removes unimportant weights, e.g., those below a certain threshold [5]. *Quantization* converts floating-point arithmetic to integer arithmetic. For example, INT8 post-training quantization [6] involves converting the weights and activations of a trained DNN model from 32-bit floating-point representations to 8-bit integer representations. Quantization-Aware Training (QAT) methods like Power-of-Two (PoT) [7] enable training with low-bit-width representations, and PoT post-training quantization on a logarithmic scale offers hardware advantages. However, while these techniques save memory and computing costs, they may introduce a high loss in accuracy.

Hardware-level optimizations include specifically designed architectures to enhance power efficiency in DNN accelerators. For instance, systolic array architecture optimizes data flow to minimize memory access [8]. [9] introduces an early-stop mechanism to skip redundant MAC operations. But this technique requires complex control logic for implementation, which

may increase system complexity. The PowerPruning technique [10] selectively prunes weights based on their magnitude and their impact on power consumption during MAC operations. However, the effectiveness of PowerPruning is closely tied to the specific dataset and network architecture used. This dependency arises because the MAC operations’ power consumption is influenced by both the input data and the trained weights, which vary across different datasets. Additionally, the model architecture determines the number of MAC invocations, further affecting power consumption. As a result, the performance of PowerPruning is limited to the specific contexts for which it is tuned.

While power consumption has been explored as a criterion for pruning [10], *to the best of our knowledge*, no existing work has focused on approximating quantized weights with attention to the power consumption of individual weight values. Existing quantization techniques have typically been developed to conserve accuracy or optimize hardware efficiency. However, a method that effectively balances both aspects remains unexplored. To overcome these limitations, *for the first time*, we propose *PowerShift*, a power-of-two approximation method tailored for INT8-quantized models to improve hardware efficiency. This approach involves approximating INT8 quantized power-hungry weights to the nearest power-of-two, ensuring that approximation errors remain within acceptable limits. Unlike PowerPruning [10], which selects weights based on dynamic power consumption tied to specific datasets and model architectures, *PowerShift* leverages the inherent properties of multiplication. We enable energy-efficient bit-shifting operations instead of conventional multiplications by approximating the trained model’s power-hungry weights as single or double power(s)-of-two. Our key contributions are as follows:

- We propose *PowerShift*, a method that approximates INT8 quantized weights to the nearest power-of-two, facilitating efficient bit-shifting operations in hardware while maintaining acceptable error margins.
- Subsequently, we extend our approach to approximate a weight using two power-of-two numbers and assess its impact on power consumption and inference accuracy. Use of two powers-of-two reduces the power consumption even further by virtue of allowing approximation of more weight values.
- To validate our approach, we have conducted experiments on LeNet-5, ResNet-20, ResNet-50, and EfficientNet-B0-Lite architectures trained on CIFAR-10, CIFAR-100, and ImageNet datasets. The *PowerShift* offers up to 4.8% reduction in power consumption relative to conventional power-aware pruning

This work was supported by the Information Security Education and Awareness (ISEA), an initiative of the Ministry of Electronics and Information Technology (MeitY), Government of India and Science and Engineering Research Board (SERB) India (project CRG/2022/003821). Vishesh Mishra was supported by Prime Minister’s Research Fellowship (PMRF) from the Ministry of Education, Government of India.

strategy [10]. Also, it provides 1.6pp (pp= percentage point¹) higher improvement in model accuracy over the baseline. Additionally, in comparison to power-of-two quantization method [7], *PowerShift* provides upto 15% higher power savings while maintaining comparable accuracy.

II. MOTIVATION

In DNN accelerators, the MAC unit significantly impacts overall power consumption during inference. The MAC unit multiplies a weight with an activation/input and adds the result to an existing partial sum, as shown in Fig. 1(a). Notably, multiplication within the MAC unit consumes approximately $6\times$ more energy than addition, making it the primary source of power consumption, as illustrated in Fig. 1(b).

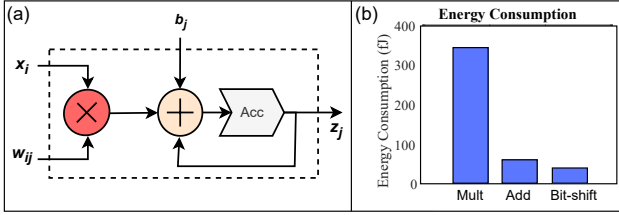


Fig. 1: MAC unit and energy consumption of its components.

Our approach directly addresses the dominant cause of power consumption by optimizing the multiplication operation itself. Traditionally, multiplying an INT8 quantized weight by an input or intermediate activation requires a full multiplication, which is resource-intensive and time-consuming. The main crux of our work is to represent the weight as a single or double power-of-two, i.e., $2^k + 2^l$ where $k \in [0, 7]$, $l \in [0, k]$. Then, the multiplication can be efficiently implemented using simple bit-shift operation, which requires a single clock cycle and minimal logic gates.

III. POWERSHIFT: APPROXIMATING WEIGHTS FOR POWER SAVINGS

In this section, we first show that bit-shifting incurs significantly less power than conventional multiplication. Building on these findings, we propose the *PowerShift* framework, which leverages the power-saving advantages of bit-shifting by approximating INT8 weights to the nearest power-of-two. We then analyze the impact of weight approximation errors introduced by *PowerShift* and explore its integration with the existing *PowerPruning* technique [10].

Power Model: We used Synopsys Power Compiler in 45nm CMOS NanGate technology and assessed static and dynamic power components. The short-circuit power is negligible and hence, ignored. The static power of the 8-bit signed multiplier is measured as $394.2 \mu W$. For each input combination, the dynamic power consumption ($P_{dynamic}$) is calculated by incorporating both switching activity and noise, accounting for process variations and environmental uncertainties. The dynamic power consumption is computed as per Equation (1), where α represents the switching activity factor, C_{load} is the

load capacitance, V_{dd} is the supply voltage, f denotes the clock frequency and η accounts for noise.

$$P_{dynamic} = \alpha \times C_{load} \times V_{dd}^2 \times f + \eta \quad (1)$$

We have used default parameter values as per 45nm tech node, i.e., $C_{load} = 1.1 \text{ pF}$, $V_{dd} = 1.3 \text{ V}$, and $f = 1 \text{ GHz}$. α ranges from $[0, 0.3]$ depending upon switching activity. To determine the noise factor η , we first calculated the minimum and maximum deviations in power consumption by varying pressure from 1.1 atm to 1.8 atm and temperature from 20°C to 120°C . Using Synopsys Power Compiler, we have found the minimum and maximum deviations to be $12.8 \mu W$ and $61.5 \mu W$, respectively. Consequently, the noise factor η was generated randomly, following a weak-sense stationary (WSS) Gaussian distribution, with values ranging from $12.8 \mu W$ to $61.5 \mu W$.

A. Bit-shifting vs conventional multiplication

We have experimented with two scenarios, as detailed below:

- ① **Conventional Multiplication:** Both input operands (Input 1 and Input 2) are varied across all possible signed 8-bit integer values, ranging from -128 to 127.
- ② **Bit-Shift Multiplication:** Here, for input combinations where one operand can be represented as a single power-of-two number (2^k), the multiplication is executed using a bit-shift operation. For all other combinations, the multiplications are performed using the conventional method.

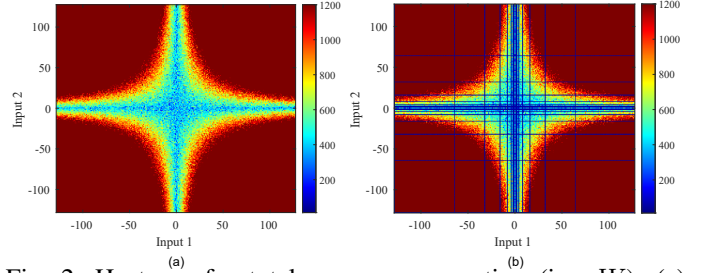


Fig. 2: Heatmap for total power consumption (in μW): (a) Conventional multiplier, (b) Bit-shift multiplier

Fig. 2(a) and Fig. 2(b) represent heatmaps of total power consumption as a function of input combinations for case ① and ②, respectively. The x-axis and y-axis correspond to the input values ranging from -128 to 127, and the color intensity represents the power consumption in μW . The heatmap demonstrates significant differences between conventional multiplication and bit-shift operations. On average, bit-shift operations exhibit $4.2\times$ lower power consumption than conventional multiplications, highlighting the efficiency of bit-shifting as a low-power alternative. Conversely, conventional multiplication showed higher and more variable power consumption, particularly for larger values of inputs, due to increased switching activity. This analysis underscores that *using power-of-two approximations during DNN inference can substantially reduce power consumption*. Based upon this analysis, we next present the proposed *PowerShift* framework in Section III-B.

B. PowerShift Framework

Given a pre-trained DNN model with INT8 quantized weights, we aim to approximate a subset of these weights to power-of-two. This approximation leverages the efficiency

¹https://en.wikipedia.org/wiki/Percentage_point

of bit-shifting operations, replacing traditional multiplication in MAC units and thus enhancing hardware performance. Consider $W \in \mathbb{R}^{n \times d}$ as the weight matrix for a given layer in

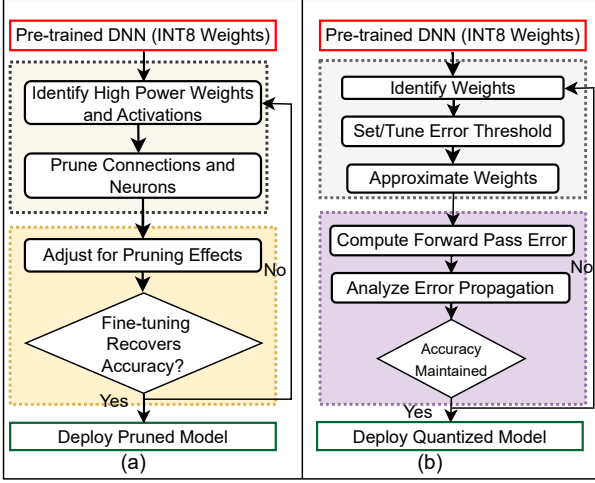


Fig. 3: Work flow of (a) PowerPruning [10] and (b) *PowerShift* the DNN, where n represents the number of input neurons and d denotes the output dimensionality of the layer. Each element $w_{ij} = \{w_{ij}^7, w_{ij}^6, w_{ij}^5, \dots, w_{ij}^0\} \in W$ signifies a weight connecting input neuron i to output neuron j and $x_i = \{x_i^7, x_i^6, x_i^5, \dots, x_i^0\}$ denotes the corresponding input activation. During inference, the multiplication operation $x_i \times w_{ij}$ (as expressed in Equation (2)) involves three stages: partial product generation, compression, and addition, which collectively make multiplication a power-consuming operation.

$$x_i \cdot w_{ij} = \left(\sum_{k=0}^6 x_i^k \cdot 2^k - x_i^7 \cdot 2^7 \right) \times \left(\sum_{k=0}^6 w_{ij}^k \cdot 2^k - w_{ij}^7 \cdot 2^7 \right) \quad (2)$$

$$= \sum_{k=0}^6 \sum_{l=0}^6 x_i^k \cdot w_{ij}^l \cdot 2^{k+l} - \sum_{k=0}^6 x_i^k \cdot w_{ij}^7 \cdot 2^{k+7}$$

$$- \sum_{l=0}^6 x_i^7 \cdot w_{ij}^l \cdot 2^{7+l} + x_i^7 \cdot w_{ij}^7 \cdot 2^{14}$$

In contrast, the *PowerShift* framework approximates a weight w_{ij} to the nearest power-of-two i.e., $w_{ij} = 2^{k_{ij}} + \delta_{ij}$ where $2^{k_{ij}}$ is the power-of-two closest to w_{ij} , and δ_{ij} is the approximation error, constrained by Equation (3). This approximation enables multiplication via bit-shifting, such that $x_i \times w_{ij}$ can be approximated as $x_i \ll k_{ij}$.

$$|\delta_{ij}| = |w_{ij} - 2^{k_{ij}}| \leq \epsilon \quad (3)$$

Note that the approximation is only applicable when the error δ_{ij} is within a predefined threshold ϵ , ensuring minimal deviation from the original weight. This way, only a subset of weights $w' \subseteq w$ undergo power-of-two approximation. The end-to-end workflow of our proposed *PowerShift* technique is depicted in Fig. 3(b).

Error Analysis: Approximating a subset of weights $w' \subseteq w$ to power-of-two introduces errors in the forward pass that can propagate through the network. To address this, we carefully analyze these errors to ensure they do not significantly impact the network's performance. The analysis is divided into two

parts: error in the forward pass and error propagation across the network, as discussed in detail below.

1. *Error in Forward Pass:* Let z_j be the output of the j^{th} layer before applying non-linearity, computed as:

$$z_j = \sum_{i=1}^n w_{ij} x_i + b_j \quad (4)$$

where b_j is the bias term and x_i represents the input activations. With *PowerShift* approximation, the output changes to:

$$z'_j = \sum_{i=1}^n 2^{k_{ij}} x_i + \sum_{i=1}^n \delta_{ij} x_i + b_j. \quad (5)$$

Here, δ_{ij} denotes the approximation error for weight w_{ij} . The error introduced in the forward pass denoted as ϵ_j is expressed using Equation (6).

$$\epsilon_j = z'_j - z_j = \sum_{i=1}^n \delta_{ij} x_i. \quad (6)$$

This error can affect the activations forwarded to subsequent layers in the network. To mitigate this impact, we constrain the magnitude of the approximation error δ_{ij} , thereby limiting ϵ_j within each layer. Specifically, the choice of δ_{ij} is critical and is typically determined through empirical testing to strike a balance between approximation precision and network performance. Details on the empirical determination of δ_{ij} are provided in Section IV.

2. *Error Propagation Across Layers:* The error ϵ_j introduced in each layer can propagate through successive layers, potentially amplified or diminished based on the network's weights. Let W_j represent the weight matrix at layer j , and ϵ_j is the error introduced at that layer. Also, $\|W_j\|$ indicates the magnitude of the weights in that layer. The cumulative error ϵ_{total} after L layers is given by: $\epsilon_{\text{total}} = \prod_{j=1}^L (\|W_j\| \cdot \epsilon_j)$. The larger the norm, the more significant the weight's impact on error propagation. Although the exact influence of ϵ_j depends on the specific input x_i and the characteristics of W_j , we can ensure that ϵ_{total} remains bounded by constraining δ_{ij} . Empirical results (discussed in Section IV) demonstrate that the errors introduced by our *PowerShift* framework are tightly controlled and have minimal impact on inference accuracy.

3. *Error Minimization:* We have further excluded certain powers of two from approximation, specifically 2^0 , 2^1 , 2^2 , and 2^3 . This is because when the weight values are very small, a minor absolute change leads to a large percentage change, causing significant errors (this exclusion is not made in dual power-of-two discussed below). To further support this decision, we have assessed the Mean Relative Error Distance (MRED) [11] for different powers-of-two approximations, as defined in Equation (7), where δ_{ij} represents the approximation error and w_{ij} is the original weight.

$$\text{MRED}(\%) = \left(\frac{|\delta_{ij}|}{|w_{ij}|} \right) \times 100 \quad (7)$$

As shown in Fig. 5(a), approximating weights to lower powers of two, such as 2^0 , 2^1 , 2^2 , and 2^3 , results in substantial errors, rendering these approximations less effective. Therefore, we focus on approximating weights to higher powers of two to keep approximation errors within acceptable limits while optimizing power efficiency. To clarify, when a weight is an

$$W_{OR} = \begin{bmatrix} 3 & -5 & 16 & -54 \\ 2 & -2 & 66 & -20 \\ 1 & 0 & -1 & -127 \\ 11 & 4 & 19 & 37 \end{bmatrix} \quad W_{1P} = \begin{bmatrix} 3 & -5 & \text{blue } 16 & -54 \\ \text{blue } 2 & -2 & \text{red } 64 & -20 \\ \text{blue } 1 & 0 & -1 & \text{red } -128 \\ 11 & \text{blue } 4 & 19 & 37 \end{bmatrix} \quad W_{2P} = \begin{bmatrix} \text{blue } 3 & -5 & \text{blue } 16 & -54 \\ \text{blue } 2 & -2 & \text{blue } 66 & -20 \\ \text{blue } 1 & 0 & -1 & \text{red } -128 \\ \text{red } 10 & \text{blue } 4 & \text{red } 18 & \text{red } 36 \end{bmatrix}$$

Fig. 4: Original weights (W_{OR}), and their single power-of-two (W_{1P}) and dual power-of-two (W_{2P}) approximations. Here, for W_{1P} , $\delta = 1$, and for W_{2P} , $\delta = 2$. Weights for which bit-shift can be done are shown in blue (no approximation) and red (approximate weights). Note that, certain weights such as $36 = 2^5 + 2^2$ use two powers-of-two approximation, while certain weights such as $-128 = -2^7$ use single power-of-two approximation.

exact power-of-two ($w = 2^k$), we realize its product using bit-shift even if k is small. However, when it is not an exact *single* power-of-two (say, $w = 2^k + \delta$ and $\delta \neq 0$), we do not approximate it and hence, realize its product using conventional multiplication operation.

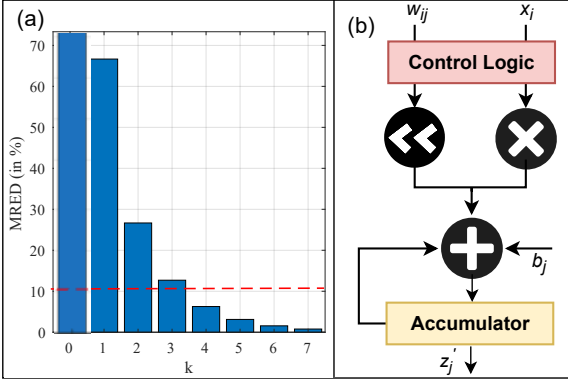


Fig. 5: (a) MRED while approximating weights to power-of-two with $|\delta_{ij}| = 1$, (b) Modified MAC unit.

4. Error Recovery via Dual Power-of-Two Representation: As δ_{ij} is increased to approximate a larger set of weight values for power savings, the forward pass error ϵ_j and total propagation error ϵ_{total} also rise. To address this issue, we extend our method to allow the approximation of weights using a combination of two power-of-two terms, i.e., $2^{k_1} + 2^{k_2}$, where k_1 and k_2 are integers such that $2^{k_1} \leq w_{ij} \leq 2^{k_1} + 2^{k_2}$, and k_1 is typically a smaller value (e.g., 1 or 2) while k_2 is selected based on the discussion provided earlier. Notably, dual power-of-two representation helps to approximate weights where the approximation error δ_i exceeds a threshold of ϵ for single power-of-two approximations. Formally, the weight w_{ij} is approximated as per Equation (8) and the approximation is applied if the condition expressed in Equation (9) is satisfied. Here in Equation (9), δ_{max} is set between 3 and 5 to allow for a broader range of approximation without significant accuracy degradation. Experimental results presented in Section IV show that extending the approximation to two powers-of-two significantly reduces the overall error without substantially increasing the power consumption, as bit-shifting remains much more efficient than traditional multiplication.

$$w_i = 2^{k_1} + 2^{k_2} + \delta_{ij}, \quad \text{where } |\delta_{ij}| \leq \epsilon \quad (8)$$

$$|w_{ij} - (2^{k_1} + 2^{k_2})| \leq \delta_{max}, \quad (9)$$

Example: An illustrative example demonstrating a 4x4 INT8 quantized matrix undergoing approximations to enable efficient inference is presented in Fig. 4. Here, the original weight matrix W_{OR} is approximated using powers-of-two representations.

In the single power-of-two (1P) format, selected values are replaced with the nearest single power-of-two, as highlighted in color: red when some error is introduced and blue when no error is introduced. The double power-of-two (2P) format (W_{2P}) further seeks to reduce error magnitude by using a sum of two powers-of-two numbers, resulting in improved approximation fidelity.

Scope of approximations: Table I shows the percentage of weights approximated by *PowerShift* for various neural networks. In the baseline, only a small fraction of weights are powers-of-two. For $(p = 1, \delta = 2)$, *PowerShift* increases the percentage of weights representable as power-of-two by nearly 4pp (pp=percentage point), whereas for $(p = 2, \delta = 4)$, it increases them between 19pp to 31pp for various networks. Evidently, with increasing p and δ , *PowerShift* performs aggressive approximation, leading to increased energy saving.

TABLE I: % of INT8 weights representable in power-of-two

Model	INT8 weights representable in power-of-two (in %)		
	Baseline	PowerShift ($p = 1, \delta = 2$)	PowerShift ($p = 2, \delta = 4$)
LeNet-5	6.3%	10.1%	27.3%
ResNet-20	6.8%	11.4%	31.6%
ResNet-50	7.4%	11.7%	38.5%
EfficientNet-B0-Lite	5.1%	9.2%	24.0%

The process of *PowerShift* is illustrated in Fig. 3(b). In this phase, we first identify the weights that can be approximated. For this purpose, we have initially set the approximation error δ_{ij} to its minimum value, i.e., $|\delta_{ij}| = 1$, to limit the approximation errors. We have subsequently conducted experiments with increasing values of δ_{ij} , and detailed results are presented in Section IV.

Architectural Support: *PowerShift* is implemented by modifying the conventional MAC unit. The standard MAC unit (refer to Fig. 1(a)) handles integer multiplication and accumulation for DNN inference using INT8 weights and inputs, consisting of a multiplier and adder connected to an accumulator. In the modified MAC architecture as shown in Fig. 5(b), a shifter and control logic are added to enable power-of-two weight approximation. The shifter replaces multiplication with bit-shifting for weights approximated to powers-of-two. The control logic determines whether to use the shifter for bit-shifted operations or the multiplier for standard multiplication based on the weight value. While the control logic adds slight overhead (refer to Section IV), fast bit-shifting compensates for it, leading to a negligible increase in inference delay. In cases where non-power-of-two weights require traditional multiplication, the delay may exceed that of the conventional MAC unit. However, since power-of-two approximation reduces the frequency of full

multiplications, the overall impact on performance is minimal.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

We have evaluated four neural networks: LeNet-5 and ResNet-20 on CIFAR-10 dataset, ResNet-50 on CIFAR-100 and EfficientNet-B0-Lite on ImageNet dataset. The evaluations are performed using a Tyrone Camarero SSA70Ti-45L system equipped with an Nvidia RTX 3080 TI 128 GB GPU. Power consumption during inference is estimated using Synopsys Power Compiler, which simulates a systolic array and executes these neural networks through Modelsim. The simulation utilizes a netlist description of the systolic array synthesized with NanGate 45nm cell library, operating at a clock frequency of 1 GHz. Five distinct experimental configurations are examined:

A. Baseline: In the baseline configuration, both weights and activations are quantized to 8-bit integers. The neural networks are trained using TensorFlow with quantization-aware training [6]. Post-training, small weights are pruned to compress the network. The model is then deployed.

B. PoT: In this configuration, we utilize the established Power-of-Two (PoT) quantization technique, as elaborated in [7]. In this work, the authors have employed logarithmic quantization with base-2 encoding, leveraging the non-uniform distribution of neural network weights. This approach restricts certain weight distribution to Power-of-Two (PoT) thus simplifying hardware implementation by replacing multiplications with bit-shift operations.

C. PowerPruning: We compare our work with a recently developed technique, viz., PowerPruning [10] (see Fig. 3(a)), using a threshold power value of 900 μ W.

D. PowerShift: After training in the baseline setting, weight values are approximated to single or double powers of two, denoted as $p = 1$ or 2. Specifically, weights are approximated based on their respective error values δ_{ij} . For $|\delta_{ij}| = 1$ or 2, a single power-of-two approximation ($p = 1$) is applied; for larger errors ($|\delta_{ij}| = 3$ or 4), a double power-of-two approximation ($p = 2$) is used. We have varied δ_{ij} starting from $|\delta_{ij}| = 1$ and observed an accuracy degradation exceeding 4.5% for $|\delta_{ij}| > 4$, so we have limited δ_{ij} in $[1, 4]$. Additionally, weights are approximated to higher powers of two, specifically $2^4, 2^5, 2^6$, and 2^7 , as detailed in Section III-B.

E. PowerPruning + PowerShift: Since our technique is complementary to PowerPruning [10], we also evaluate their combination, where PowerPruning is applied first, followed by *PowerShift*. This integration first selects weight values and activations that minimize power consumption, and then approximates them to the nearest power-of-two, compounding the overall power reduction.

B. Accuracy vs Power Consumption

The *PowerShift* approach ($p = 2, \delta = 4$) significantly enhances power efficiency compared to the baseline and traditional PoT quantization [7], as shown in Table II. For LeNet-5, *PowerShift* achieves a 51.2% power reduction versus 36.9% for PoT, with a modest accuracy degradation of 1.2% compared to 0.7%. Similar trends are observed in ResNet-20

(55.7% power reduction vs. 40.2%, 1.4% degradation vs. 0.8%), ResNet-50 (48.1% vs. 31.6%, 0.9% vs. 0.6%), and EfficientNet-B0-Lite (27.9% vs. 10.3%, 0.6% vs. 0.4%). *PowerShift* thus offers superior power savings, enabling efficient shift-based computations while maintaining competitive accuracy. Overall, in comparison with power-of-two quantization method [7], *PowerShift* provides upto 15% higher power savings while maintaining comparable accuracy.

The *PowerShift* approach (configuration D) outperforms *PowerPruning* [10] in terms of lower accuracy degradation and higher power reduction, as illustrated in Table II. For LeNet-5, *PowerShift* shows 1.2% accuracy degradation compared to 2.3% for *PowerPruning*, with 51.3% power reduction versus 44.6%. In ResNet-20, *PowerShift* achieves 1.4% degradation against 3%, and 51.5% power savings over 50.7%. For ResNet-50, *PowerShift* offers 0.9% degradation (vs. 1.5%) and 48.7% reduction (vs. 46.2%). Similarly, in EfficientNet-B0-Lite, *PowerShift* yields 0.6% degradation versus 1.5%, and 31.5% power reduction against 27.0%, demonstrating *PowerShift*'s superior balance of efficiency and performance. On an average, *PowerShift* achieves up 4.8% higher power savings compared to existing power-aware pruning techniques, while also enabling up to 1.6% higher model accuracy, highlighting the synergistic benefits for greater energy efficiency.

Combining *PowerPruning* with *PowerShift* (configuration E) enhances power savings compared to *PowerPruning* alone [10], albeit with slightly higher accuracy degradation, as shown in Table II. For LeNet-5, the combination results in 67.4% power reduction (vs. 44.6% for *PowerPruning*), with 2.9% degradation against 2.3%. In ResNet-20, it achieves 61.5% savings (vs. 50.7%), with 3.7% degradation compared to 3%. For ResNet-50, the combination yields 68.2% reduction (vs. 46.2%), with 2% degradation versus 1.5%. In EfficientNet-B0-Lite, it provides 50.9% power savings (vs. 27.0%), with 1.9% degradation against 1.5%.

Fig. 6 presents the power savings (in %) and accuracy degradation (in pp) for selected model–dataset pairs under different approximation configurations, characterized by varying values of δ_{ij} and p . Specifically, the results are shown for $p = 1, \delta = 2$ and for $p = 2, \delta = 4$. As shown in Fig. 6, the proposed *PowerShift* technique consistently achieves the highest power savings across all cases. The next best performance is obtained by the *PowerPruning* (PwP) approach. Although PoT quantization reduces power consumption relative to the baseline, it still incurs higher overhead compared to both the proposed *PowerShift* method and the existing *PowerPruning* technique. Further, *PowerShift* offers second-best accuracy degradation after PoT technique.

Fig. 7 presents four scatter subplots illustrating the trade-off between accuracy degradation and power savings. Different markers are used to distinguish the techniques: circles for PoT, squares for PwP, triangles for *PowerShift* ($p = 1, \delta = 2$), and diamonds for *PowerShift* ($p = 2, \delta = 4$). The proposed *PowerShift* consistently demonstrates the most favorable trade-off characteristics, achieving superior power savings while maintaining minimal accuracy degradation. Overall, when com-

TABLE II: Comparison of various techniques (PwP=PowerPruning, pp= percentage point). PowerShift uses $p = 2, \delta = 4$.

	Baseline results		Accuracy degradation ↓				% Saving in $P_{dynamic}$ ↑			
	Accuracy	$P_{dynamic}$ (mW)	PoT	PwP	PowerShift	PwP+PowerShift	PoT	PwP	PowerShift	PwP+PowerShift
LeNet-5	80.7%	288.3	0.7pp	2.3pp	1.2pp	2.9pp	36.90%	44.60%	51.30%	67.40%
ResNet-20	91.9%	473.1	0.8pp	3pp	1.4pp	3.7pp	40.20%	50.70%	51.50%	61.50%
ResNet-50	79.9%	511.7	0.6pp	1.5pp	0.9pp	2.0pp	31.60%	46.20%	48.70%	68.20%
EfficientNet-B0	74.4%	154.8	0.4pp	1.5pp	0.6pp	1.9pp	10.30%	27.00%	31.50%	50.90%

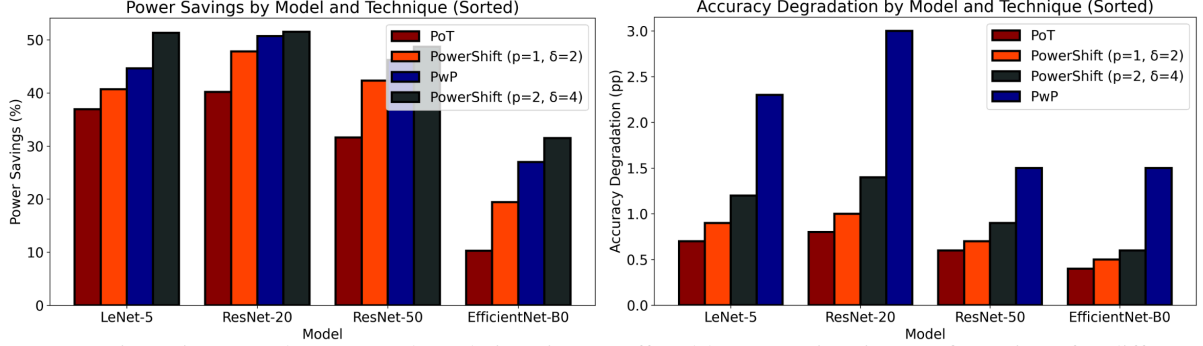


Fig. 6: Power savings (in %) and accuracy degradation (in pp) offered by approximation configurations for different models.

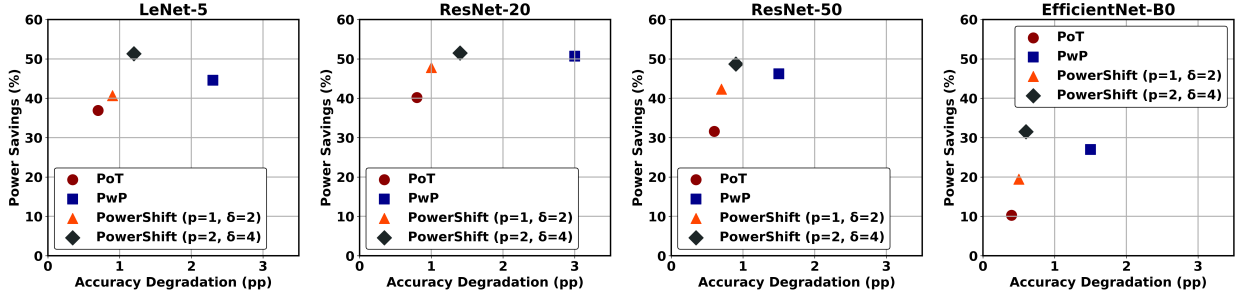


Fig. 7: Power savings vs accuracy degradation tradeoff for different model-dataset and approximation configurations.

pared with state-of-the-art power-of-two quantization method (TinyML'22), *PowerShift* provides upto 15% higher power savings while maintaining comparable accuracy.

C. Hardware Evaluation

PowerShift modifies the conventional MAC unit to exploit power-of-two weights through bit-shifting. To assess hardware costs, we implemented both baseline and modified MAC architectures in Verilog HDL and synthesized them using Synopsys Design Compiler with 45nm Open Cell Library. Table III reports the area, power, and delay for 100 consecutive MAC operations with weights drawn from the LeNet-5 model trained on CIFAR-10, where 22 operations involved power-of-two weights. The results show that bit-shifting in *PowerShift* yields substantial energy savings of 40.7% with only a modest area overhead of 3.8%.

TABLE III: Hardware evaluation of modified MAC

MAC Unit	Area (μm^2)	Power (μW)	Delay (ns)	Energy (pJ)
Conventional	584.6	37806.9	112.8	4264.6
Modified	606.8	21752.7	116.1	2525.4

V. CONCLUSION

This paper introduces *PowerShift*, a method for approximating INT8 weights to powers-of-two, enhancing power efficiency in DNN inference through bit-shifting. We integrate *PowerShift* with PowerPruning, which reduces network complexity by

pruning high-impact weights and activations. This combined approach optimizes power savings while preserving accuracy. Experimental results confirm that approximating weights to higher powers of two minimizes approximation errors. Our approach effectively balances power efficiency and performance, offering a robust solution for power-constrained environments.

REFERENCES

- [1] F. Yu *et al.*, "Rein the robots: Robust dnn-based image recognition in autonomous driving systems," *IEEE TCAD*, 2020.
- [2] J. Mao *et al.*, "MoDNN: Local distributed mobile computing system for deep neural network," in *DATE*, 2017.
- [3] V. Mishra *et al.*, "Art-mac: Approximate rounding and truncation based mac unit for fault-tolerant applications," in *ISCAS*. IEEE, 2022.
- [4] K. Zhao, S. Huang, P. Pan, Y. Li, Y. Zhang, Z. Gu, and Y. Xu, "Distribution adaptive int8 quantization for training cnns," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 4, 2021, pp. 3483–3491.
- [5] S. Han *et al.*, "Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding," *ICLR*, 2015.
- [6] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *CVPR*, 2018.
- [7] D. Przewlocka-Rus *et al.*, "Power-of-two quantization for low bitwidth and hardware compliant neural networks," *tinyML*, 2022.
- [8] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *ISCA*, 2017.
- [9] V. Akhlaghi *et al.*, "Snapea: Predictive early activation for reducing computation in deep convolutional neural networks," in *ISCA*, 2018.
- [10] R. Petri *et al.*, "Powerpruning: Selecting weights and activations for power-efficient neural network acceleration," in *DAC*, 2023.
- [11] J. Liang *et al.*, "New metrics for the reliability of approximate and probabilistic adders," *IEEE Transactions on computers (TC)*, 2012.